

Classification Metrics, Worked Out by Hand

From a confusion matrix to precision, recall, F1 and accuracy

A complete numerical walk-through of how four counts — true positives, false positives, false negatives, true negatives — turn into every headline number a classifier reports. We fix one small binary confusion matrix, derive precision, recall, F_1 and accuracy by hand, then show *why accuracy lies* under class imbalance, and finally extend to a 3×3 multi-class matrix to compute macro- and micro- F_1 exactly. Every number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It takes the course’s own confusion-matrix machinery (the `evaluate_binary` loop from “Proper Evaluation Loops”) and slows it to a crawl: one binary example with $N = 100$ predictions split into $TP = 40$, $FP = 10$, $FN = 15$, $TN = 35$, then a 3×3 multi-class case. We compute each metric digit by digit and check it two ways.

How to read this. Each metric gets its own derivation and a coloured box that says *when* it is the one to optimise. The running thread is a fraud / disease-style positive class that is rare — exactly the setting where the course warns “accuracy is misleading”. Read this before the PR/ROC topic; those curves are just these same four counts swept across a threshold.

Concepts covered, in order: the confusion matrix and its four counts · precision = $TP/(TP+FP)$ · recall = $TP/(TP+FN)$ · the F_1 harmonic mean · accuracy and why it misleads under imbalance · per-class metrics from a 3×3 matrix · macro- F_1 (unweighted mean) versus micro- F_1 (pooled counts) · which metric to report when.

1 The setup: four counts in a 2×2 box

A trained binary classifier turns each example into a hard label $\hat{y} \in \{0, 1\}$ (positive = 1). Compared against the truth y , every prediction lands in exactly one of four cells. That table is the **confusion matrix**, and *all* the metrics in this document are algebra on its four counts.

		Predicted	
		$\hat{y} = 1$ (pos)	$\hat{y} = 0$ (neg)
Actual	$y = 1$ (pos)	TP = 40	FN = 15
	$y = 0$ (neg)	FP = 10	TN = 35

The four names read as “(*true/false*) = was the prediction right?”, and “(*positive/negative*) = what did the model *say*?”. So a **false positive** is the model crying “positive” when the truth is negative; a **false negative** is a positive that slipped through. Our totals: $N = TP + FP + FN + TN = 40 + 10 + 15 + 35 = 100$, with $TP + FN = 55$ actual positives and $FP + TN = 45$ actual negatives.

$$\text{counts} = \begin{bmatrix} TP & FN \\ FP & TN \end{bmatrix} = \begin{bmatrix} 40 & 15 \\ 10 & 35 \end{bmatrix}.$$

Intuition

Two metrics read this box along two different edges. **Precision** reads down the predicted-positive *column*: “of everything I *flagged*, how many were actually right?” **Recall** reads across the actual-positive *row*: “of everything that *was* positive, how many did I *catch*?” Precision is judged by the accuser; recall is judged by the victims. They optimise against each other, which is why one number (F_1) tries to summarise both.

2 Step 1 — Precision: of the flagged, how many were right?

Precision divides the correctly-flagged positives by *everything you flagged* (the predicted-positive column, $TP + FP$):

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{40}{40 + 10} = \frac{40}{50} = 0.800.$$

So 80.0% of the examples this model called positive truly were. The 10 false positives are the price: one in five alarms is wrong. Precision never looks at FN or TN — missed positives and correctly-rejected negatives are simply invisible to it.

When this is the right choice

Precision-heavy: spam filtering. A false positive here means a real, wanted email is silently dumped in the spam folder — a far worse outcome for the user than the occasional spam that sneaks into the inbox (a false negative). You want near-certainty before you act, so you tune the threshold up and accept lower recall. Same logic for “is this transaction definitely fraud, freeze the card?” where a wrong freeze angers a good customer.

3 Step 2 — Recall: of the truly positive, how many were caught?

Recall (a.k.a. sensitivity or true-positive rate) divides the correctly-caught positives by *all actual positives* (the actual-positive row, TP + FN):

$$\boxed{\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}} = \frac{40}{40 + 15} = \frac{40}{55} = 0.727.$$

The model catches 72.7% of the real positives; the other $1 - 0.727 = 0.273$ (15 of 55) are missed — those are the false negatives. Recall never looks at FP or TN: you can buy perfect recall by flagging *everything*, which is exactly why recall alone is not enough.

When this is the right choice

Recall-heavy: disease screening. A false negative is a sick patient sent home untreated — potentially fatal — whereas a false positive merely triggers a second, more specific test. The cost is wildly asymmetric, so a screening test is tuned for high recall (catch every possible case) and lets precision fall; the follow-up test cleans up the false alarms. Fraud *detection* and safety recalls follow the same “miss nothing” logic.

Watch out

Precision and recall trade off through the decision threshold, not independently. Lowering the threshold flags more examples: recall rises (you catch more real positives) but precision usually falls (more false alarms). Quoting one without the other — or without the threshold — is meaningless. The PR curve in the next topic is precisely this pair traced as the threshold moves.

4 Step 3 — F_1 : the harmonic mean of precision and recall

To collapse precision P and recall R into one number we use their *harmonic* mean, not the arithmetic one. The harmonic mean punishes imbalance: it stays close to the *smaller* of the two, so you cannot fake a good F_1 by maxing one metric and tanking the other.

$$\boxed{F_1 = \frac{2PR}{P + R}} = \frac{2(0.800)(0.727)}{0.800 + 0.727} = \frac{1.164}{1.527} = 0.762.$$

There is an equivalent form straight from the counts, handy because it never touches P or R separately:

$$F_1 = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}} = \frac{80}{80 + 10 + 15} = \frac{80}{105} = 0.762. \quad \checkmark$$

Both routes give 0.762, sitting between $R = 0.727$ and $P = 0.800$ but pulled toward the smaller. (The arithmetic mean would have been 0.764 — barely different here, but the gap explodes when P and R diverge.)

Mini-example — this operation in isolation

Why *harmonic*? Take a lopsided classifier with $P = 0.95$, $R = 0.10$ (flags almost nothing, but is right when it does). Arithmetic mean = 0.525 — looks mediocre-but-usable. Harmonic mean: $F_1 = 2(0.95)(0.10)/(0.95 + 0.10) = 0.190/1.05 = 0.181$. The F_1 correctly screams “this

model misses 90% of positives.” The harmonic mean refuses to let a single strong number paper over a weak partner.

Watch out

F_1 **ignores** TN **entirely** — it is built from TP, FP, FN only. That is a *feature* for rare-positive problems (the millions of correctly-ignored negatives should not inflate your score) but means F_1 says nothing about how well you handle the negative class. If both classes matter, report per-class metrics or macro- F_1 (Step 6), not a single positive-class F_1 .

5 Step 4 — Accuracy, and why it misleads under imbalance

Accuracy is the fraction of *all* predictions that were correct — the diagonal of the box over the total:

$$\boxed{\text{Accuracy} = \frac{\text{TP} + \text{TN}}{N}} = \frac{40 + 35}{100} = \frac{75}{100} = 0.750.$$

On our balanced-ish example (55 positives, 45 negatives) accuracy = 0.750 is a reasonable summary. The danger appears when one class dominates.

The imbalance trap. Suppose the positive class is rare: $N = 100$ with 95 negatives and only 5 positives. Build the laziest possible model — it *predicts negative for everything*. Its confusion matrix is

$$\begin{aligned} \text{TP} &= 0, & \text{FP} &= 0, & \text{FN} &= 5, & \text{TN} &= 95, \\ \text{Accuracy} &= \frac{0 + 95}{100} = 0.950, & \text{Recall} &= \frac{0}{0 + 5} = 0.000. \end{aligned}$$

A model with 95.0% accuracy that has *never once* caught a positive — recall 0.000, precision undefined (0/0), $F_1 = 0.000$. Accuracy rewarded it for the 95 easy negatives and never noticed it solved nothing. This is exactly the course’s warning: “a model that predicts *no fraud* for every sample achieves ~95% accuracy.”

Watch out

Never report accuracy alone on imbalanced data. The majority-class baseline accuracy equals the majority fraction (0.95 here), so any accuracy below that is worse than doing nothing, and accuracy just above it can still mean recall = 0. Always pair it with precision, recall and F_1 on the *minority* (positive) class — the numbers that actually move when the model learns something.

Intuition

Accuracy weights every example equally, so it is dominated by whichever class is most common. Precision/recall/ F_1 deliberately *exclude* TN, which is the cell imbalance inflates. That single design choice is why the minority-class F_1 tracks real skill while accuracy tracks the class ratio.

6 Step 5 — Per-class metrics from a 3×3 matrix

With $K > 2$ classes the confusion matrix is $K \times K$: entry C_{ij} counts examples whose *true* class is i but were *predicted* as j . The diagonal is correct; everything off-diagonal is an error. Here is a 3-class matrix over $N = 80$ examples (rows = true, columns = predicted):

$$C = \begin{bmatrix} 25 & 3 & 2 \\ 8 & 14 & 3 \\ 4 & 6 & 15 \end{bmatrix}, \quad \begin{array}{l} \text{row sums (true)} = (30, 25, 25) \\ \text{col sums (pred)} = (37, 23, 20) \\ \text{diagonal (correct)} = (25, 14, 15), \text{ sum } 54 \end{array}$$

Each class becomes its own one-vs-rest binary problem. For class c : $\text{TP}_c = C_{cc}$; $\text{FP}_c = (\text{column } c \text{ sum}) - \text{TP}_c$ (others predicted as c); $\text{FN}_c = (\text{row } c \text{ sum}) - \text{TP}_c$ (class c predicted as something else). Then the same precision/recall/ F_1 as before.

Class 0: $\text{TP} = 25$, $\text{FP} = 37 - 25 = 12$, $\text{FN} = 30 - 25 = 5$.

$$P_0 = \frac{25}{37} = 0.676, \quad R_0 = \frac{25}{30} = 0.833, \quad F_{1,0} = \frac{2(0.676)(0.833)}{0.676+0.833} = 0.746.$$

Class 1: $\text{TP} = 14$, $\text{FP} = 23 - 14 = 9$, $\text{FN} = 25 - 14 = 11$.

$$P_1 = \frac{14}{23} = 0.609, \quad R_1 = \frac{14}{25} = 0.560, \quad F_{1,1} = 0.583.$$

Class 2: $\text{TP} = 15$, $\text{FP} = 20 - 15 = 5$, $\text{FN} = 25 - 15 = 10$.

$$P_2 = \frac{15}{20} = 0.750, \quad R_2 = \frac{15}{25} = 0.600, \quad F_{1,2} = 0.667.$$

Class	TP	FP	FN	Precision	Recall
0	25	12	5	0.676	0.833
1	14	9	11	0.609	0.560
2	15	5	10	0.750	0.600

Note class 0 has high recall but lower precision (it over-predicts class 0: 12 false positives), while class 2 is the reverse (precise but misses 10). Per-class numbers expose exactly which class the model confuses — a single accuracy number would have hidden all of it.

7 Step 6 — Macro- F_1 versus micro- F_1

To report *one* multi-class number you must choose how to average. The two standard choices answer different questions.

Macro- F_1 — average the per-class F_1 scores with *equal weight*, no matter how rare each class is:

$$\boxed{\text{macro-}F_1 = \frac{1}{K} \sum_c F_{1,c}} = \frac{0.746 + 0.583 + 0.667}{3} = \frac{1.996}{3} = 0.665.$$

Micro- F_1 — *pool* the counts across all classes first, then compute one precision/recall/ F_1 . Summing over classes: $\text{TP} = 54$, $\text{FP} = \sum_c \text{FP}_c = 12 + 9 + 5 = 26$, $\text{FN} = \sum_c \text{FN}_c = 5 + 11 + 10 = 26$.

$$\text{micro-}P = \frac{54}{54+26} = \frac{54}{80} = 0.675, \quad \text{micro-}R = \frac{54}{54+26} = 0.675,$$

$$\text{micro-}F_1 = \frac{2(0.675)(0.675)}{0.675 + 0.675} = 0.675.$$

In single-label classification every error is simultaneously one class’s FP and another’s FN, so $\sum \text{FP} = \sum \text{FN}$ always, which forces micro-precision = micro-recall = micro- F_1 = *accuracy* = $54/80 = 0.675$. Micro- F_1 *is* accuracy in disguise here.

They disagree: macro = 0.665 vs micro = 0.675. The gap is the whole point. Macro gives the struggling class 1 ($F_1 = 0.583$) the *same* vote as the strong class 0, dragging the average down. Micro lets the larger, easier classes dominate, so it rides higher.

When this is the right choice

Choose macro when small classes matter — rare-disease subtypes, minority languages, any setting where failing a small class is unacceptable. Macro forces you to earn every class.
Choose micro (= accuracy) when every example counts the same regardless of class — e.g. raw throughput correctness on a naturally balanced set. “Weighted” F_1 (mean weighted by class support) sits between the two.

Watch out

Decide macro-vs-micro *before* you look at results, and state which you used. On imbalanced data the two can differ by a lot, and quietly reporting whichever is higher is a classic way to oversell a model. A common error: calling micro- F_1 “the F_1 ” on a multi-class problem when it is really just accuracy and tells you nothing extra.

8 All metrics, at a glance

Metric	Formula	Reads	Our value
Precision	$\text{TP}/(\text{TP}+\text{FP})$	flagged $\rightarrow$ right	0.800
Recall	$\text{TP}/(\text{TP}+\text{FN})$	positives $\rightarrow$ caught	0.727
F_1	$2PR/(P+R)$	harmonic mean	0.762
Accuracy	$(\text{TP}+\text{TN})/N$	all $\rightarrow$ right	0.750
<i>Imbalance trap (95 neg / 5 pos, predict-all-negative)</i>			
Accuracy	$(0+95)/100$	misleading	0.950
Recall	$0/(0+5)$	truth	0.000
<i>Multi-class 3×3 ($N = 80$)</i>			
Macro- F_1	$\frac{1}{K} \sum_c F_{1,c}$	all classes equal	0.665
Micro- F_1	$2P_\mu R_\mu / (P_\mu + R_\mu)$	= accuracy	0.675

These eight numbers are the entire vocabulary of classification reporting. Print the confusion matrix first whenever a model “looks good”; every metric here is just one slice of it, and the imbalance row is why you never stop at accuracy.

9 Equation cheat-sheet

Precision	$P = \frac{TP}{TP + FP}$
Recall (sensitivity / TPR)	$R = \frac{TP}{TP + FN}$
F_1 (from P, R)	$F_1 = \frac{2PR}{P + R}$
F_1 (from counts)	$F_1 = \frac{2TP}{2TP + FP + FN}$
Accuracy	$\frac{TP + TN}{N}$
Per-class (one-vs-rest)	$TP_c = C_{cc}, FP_c = \sum_{i \neq c} C_{ic}, FN_c = \sum_{j \neq c} C_{cj}$
Macro- F_1	$\frac{1}{K} \sum_c F_{1,c}$ (every class equal)
Micro- F_1	pool TP, FP, FN over classes $\Rightarrow$ = accuracy

Appendix A Reference implementation

Every number above is reproducible from the following program. The pure-NumPy block mirrors the course's confusion-matrix arithmetic; the torch block confirms the binary counts with tensor masks exactly as `evaluate_binary` does.

```
import numpy as np

# ----- binary: TP=40 FP=10 FN=15 TN=35, N=100 -----
TP, FP, FN, TN = 40, 10, 15, 35
N = TP + FP + FN + TN                # 100
precision = TP / (TP + FP)           # 0.800
recall    = TP / (TP + FN)           # 40/55 = 0.727
f1        = 2 * precision * recall / (precision + recall) # 0.762
f1_counts = 2 * TP / (2 * TP + FP + FN) # 80/105 = 0.762 (same)
accuracy  = (TP + TN) / N             # 0.750
print(round(precision,3), round(recall,3), round(f1,3), round(accuracy,3))

# ----- imbalance trap: 95 neg / 5 pos, predict all-negative -----
TP2, FP2, FN2, TN2 = 0, 0, 5, 95
acc_lazy    = (TP2 + TN2) / 100       # 0.950 (looks great)
recall_lazy = TP2 / (TP2 + FN2)       # 0.000 (caught nothing)
print(round(acc_lazy,3), round(recall_lazy,3)) # 0.95 0.0

# ----- multi-class 3x3 (rows=true, cols=pred) -----
C = np.array([[25, 3, 2],
              [ 8,14, 3],
              [ 4, 6,15]], dtype=float)
col = C.sum(axis=0) # predicted totals: [37, 23, 20]
row = C.sum(axis=1) # true totals:      [30, 25, 25]

P, R, F = [], [], []
for c in range(3):
    tp = C[c, c]
    fp = col[c] - tp
    fn = row[c] - tp
    p  = tp / (tp + fp)
    r  = tp / (tp + fn)
    P.append(p); R.append(r); F.append(2 * p * r / (p + r))
# per class F1: [0.746, 0.583, 0.667]

macro_f1 = np.mean(F)                # 0.665 (every class equal)

TPp = np.trace(C)                    # 54
FPp = sum(col[c] - C[c, c] for c in range(3)) # 26
FNp = sum(row[c] - C[c, c] for c in range(3)) # 26
micro_p = TPp / (TPp + FPp)
micro_r = TPp / (TPp + FNp)
micro_f1 = 2 * micro_p * micro_r / (micro_p + micro_r) # 0.675 == accuracy
print(round(macro_f1,3), round(micro_f1,3)) # 0.665 0.675

# ----- torch: confirm the binary counts with masks -----
import torch
y = torch.cat([torch.ones(TP+FN), torch.zeros(FP+TN)]) # 55 pos, 45 neg
preds = torch.cat([torch.ones(TP), torch.zeros(FN),
                  torch.ones(FP), torch.zeros(TN)])
tp = ((preds==1)&(y==1)).sum().item() # 40
fp = ((preds==1)&(y==0)).sum().item() # 10
```



```
fn = ((preds==0)&(y==1)).sum().item()    # 15
tn = ((preds==0)&(y==0)).sum().item()    # 35
print(tp/(tp+fp), tp/(tp+fn), (tp+tn)/100)  # 0.8 0.727... 0.75
```

— end of document —